

Hibernate a fondo: decisiones de mapeo y ciclo de vida

6. Hibernate a fondo: decisiones de mapeo y ciclo de vida

Clase persistente y formas de declarar el mapeo

Una clase persistente (o entidad) es una clase Java cuyos objetos el ORM sabe guardar y recuperar de la base de datos. Hay dos formas históricas de declarar el mapeo:

Fichero XML

Un documento aparte: `<hibernate-mapping>`, un `<class>` por entidad y un `<property>` por campo.
Legado: apenas se usa en proyectos nuevos.

Anotaciones

Directamente sobre la clase (`@Entity`, `@Column...`).
Es el estándar actual y el que usas desde la Actividad 1.1.

Por qué ganaron las anotaciones

Viven pegadas al propio código: cambias un campo y su mapeo a la vez, en el mismo fichero, sin mantener sincronizados un `.java` y un `.xml`.

Los estados de un objeto en el ORM

Transient

Recién creado con new.
Hibernate no sabe que existe.

Managed

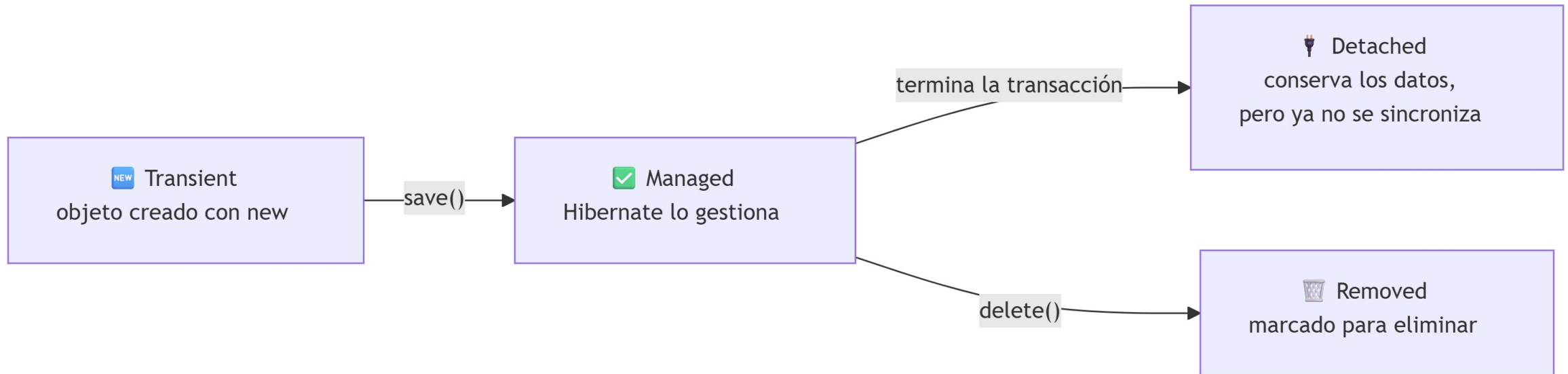
Hibernate lo gestiona: si cambias un campo, lo detecta y lo guarda al terminar.

Detached

Ya no lo gestiona:
conserva los datos,
pero los cambios no se sincronizan.

Removed

Marcado para eliminarse de la base de datos.



Crear y modificar una entidad

Crear una entidad nueva

```
Videojuego videojuego = new Videojuego();  
// TRANSIENT: solo existe en memoria  
videojuego.setTitulo("Celeste");  
  
Videojuego guardado =  
    videojuegoRepository.save(videojuego);
```

Recoge siempre lo que devuelve save()

Al guardar, Hibernate pasa a gestionarlo y lo inserta. El objeto devuelto tiene el estado definitivo, incluido el id generado.

Recuperar y modificar

```
@Transactional  
public void cambiarPrecio(Long id,  
    BigDecimal nuevoPrecio) {  
    Videojuego videojuego = videojuegoRepository  
        .findById(id).orElseThrow();  
  
    videojuego.setPrecio(nuevoPrecio);  
}
```

Managed durante la transacción

Hibernate detecta el cambio de precio y lo guarda al terminar, sin un nuevo save().

Cuando la transacción termina y Hibernate deja de gestionar el objeto, pasa a detached: conserva sus datos, pero los cambios ya no se sincronizan.

Eliminar, y cuándo deja de estar managed

Eliminar una entidad

```
@Transactional
public void eliminar(Long id) {
    Videojuego videojuego = videojuegoRepository
        .findById(id).orElseThrow();

    videojuegoRepository.delete(videojuego);
}
```



delete() → removed

Hibernate la marca para eliminarla de la base de datos al confirmar la transacción.

Contexto de persistencia

Dentro, el objeto es managed y Hibernate vigila sus cambios. Con `@Transactional`, suele estar dentro toda la transacción.

Los cuatro estados, de un vistazo: transient → managed → detached / removed.

save() por dentro, y «instalar» Hibernate

persist() y merge(), ocultos tras save()

JPA distingue guardar una entidad nueva (persist) y una que ya existía (merge). Spring Data JPA oculta esa diferencia detrás de save().

Regla 1

Si cargas una entidad dentro de una transacción y la modificas, Hibernate detecta los cambios.

Regla 2

Al llamar a save(), usa el objeto que devuelve el repository.

«Instalar» Hibernate no es un paso manual

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-data-jpa
  </artifactId>
</dependency>
```

spring-boot-starter-data-jpa trae Hibernate como implementación de JPA por defecto, además de Spring Data JPA. Ya lo tienes en tu pom.xml desde la Actividad 1.1.

GenerationType: cinco formas de generar el id

Estrategia	Cómo genera el id	Cuándo conviene
IDENTITY	Delega en una columna autoincremental de la propia base de datos.	La que usas en Videojuego y Estudio, con ids Long.
SEQUENCE	Una secuencia de la base de datos que Hibernate consulta antes de insertar.	Cuando necesitas reservar ids por adelantado.
TABLE	Simula una secuencia con una tabla auxiliar que gestiona Hibernate.	Cuando el gestor no tiene identity ni secuencias.
UUID	Genera un identificador UUID (550e8400-e29b-...).	Ids únicos sin depender de una secuencia central.
AUTO	Deja que Hibernate elija la más adecuada.	Cuando prefieres delegar la elección en el ORM.

En este proyecto seguirás con IDENTITY: tus entidades tienen un id numérico Long y PostgreSQL puede generarlo automáticamente.

TABLE y UUID, en concreto

TABLE: una tabla normal, SELECT + UPDATE por id

```
CREATE TABLE hibernate_sequence (  
    entidad VARCHAR(255) PRIMARY KEY,  
    valor_actual BIGINT  
);  
  
-- 1. lee  
SELECT valor_actual FROM hibernate_sequence  
WHERE entidad = 'videojuego';  
-- 2. incrementa  
UPDATE hibernate_sequence  
SET valor_actual = valor_actual + 1  
WHERE entidad = 'videojuego';
```

UUID: exige declarar el id como UUID

```
@Id  
@GeneratedValue(strategy =  
    GenerationType.UUID)  
private UUID id;
```

Por qué TABLE es la menos eficiente

Hace lo mismo que SEQUENCE, pero es una tabla corriente: ese SELECT/UPDATE se bloquea como cualquier fila.

Con muchas inserciones simultáneas, TABLE se convierte en un cuello de botella; SEQUENCE es un objeto del motor, optimizado para que no se bloqueen entre sí.

@Column(precision, scale) y el dinero

BigDecimal sin precision/scale

Hibernate elige un tipo de columna genérico: puede admitir más decimales de los que tiene sentido para un precio.

BigDecimal con precision/scale

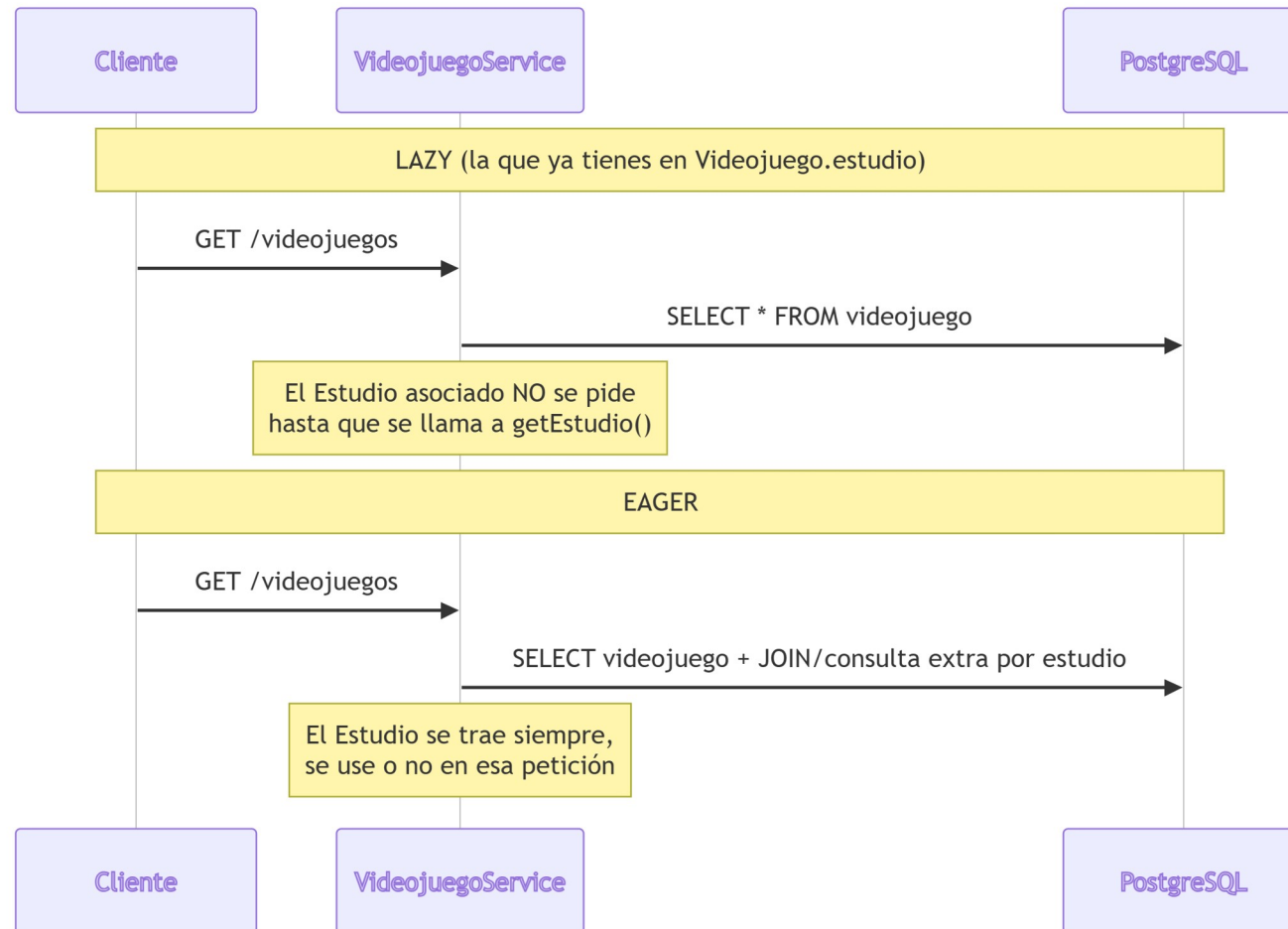
Tú fijas cuántos dígitos y decimales admite la columna. La elegida: precisión explícita, sin depender del motor.

Entero en céntimos (Long)

250099 = 2500,99 €. Elimina el problema de raíz, a cambio de más código al convertir a euros.

Compruébalo: quita temporalmente @Column(precision = 10, scale = 2) de precio, borra la tabla y reinicia con show-sql: true. El create table generará un tipo de columna distinto. En dinero, esa precisión no es decorativa.

FetchType.LAZY frente a EAGER



LAZY

No trae el Estudio asociado hasta que de verdad lo pides.

EAGER

Lo trae siempre, en la misma consulta o en una adicional: más consultas por petición aunque el cliente nunca mire ese dato.

Compruébalo con
logging.level.org.hibernate.SQL:
DEBUG y cuenta las consultas de
GET /api/v1/videojuegos.

cascade: qué otras opciones había

Valor de cascade	Qué propaga de Estudio hacia sus Videojuego
(sin cascade)	Nada: cada operación sobre un Videojuego la harías tú, una a una
PERSIST	Guardar el Estudio guarda también los videojuegos nuevos que le hayas añadido
MERGE	Actualizar el Estudio sincroniza también los cambios de sus videojuegos existentes
REMOVE	Borrar el Estudio borra también todos sus videojuegos
REFRESH / DETACH	Recargar o desvincular el Estudio hace lo mismo con sus videojuegos
ALL (la elegida)	Las cinco anteriores combinadas, en una sola palabra

Se eligió ALL porque un videojuego sin su estudio no tiene sentido de negocio. Con cascade = {PERSIST, MERGE} sin REMOVE, borrar un Estudio con videojuegos fallaría por la clave foránea.

orphanRemoval, y dónde se declara la cascada

Si orphanRemoval es...	Al quitar un Videojuego de la lista (sin borrar el Estudio)
true (la elegida)	Hibernate lo detecta como huérfano y lo elimina de la base de datos
false (por defecto)	El videojuego se queda en la base de datos, sin ningún Estudio que lo referencie

orphanRemoval = true se usa cuando el hijo no tiene sentido por separado: si deja de pertenecer al padre, también debe desaparecer de la base de datos.

La cascada va en el lado Estudio → Videojuego

El lado «uno» de la relación: al revés no tendría sentido, porque borrar un solo videojuego no debería llevarse por delante el estudio entero.

Configuración del ORM, no del conector

Conector

Cómo se conecta: usuario, contraseña, URL.

ORM

Qué hace Hibernate con tus entidades (crear tablas, validarlas, no tocar nada): ddl-auto.

Propiedad	Qué controla
<code>spring.jpa.hibernate.ddl-auto</code>	Si Hibernate crea, actualiza o solo valida las tablas al arrancar.
<code>spring.jpa.show-sql</code>	Si imprime en consola el SQL real que ejecuta por debajo.
<code>spring.jpa.properties.hibernate.format_sql</code>	Si ese SQL impreso se formatea en varias líneas.

Para mapear tipos sin correspondencia directa (cifrar un campo, dar formato propio) Hibernate necesita ayuda: un `@Converter` (`AttributeConverter`). No hace falta escribir uno todavía.

El otro lado de lo automático: data.sql

ddl-auto crea la estructura; data.sql, los datos iniciales

```
INSERT INTO estudio (nombre)
VALUES ('Nintendo');
INSERT INTO estudio (nombre)
VALUES ('Supergiant Games');
```

Con PostgreSQL hacen falta dos propiedades más

```
spring:
  jpa:
    hibernate:
      ddl-auto: update
      defer-datasource-initialization: true
  sql:
    init:
      mode: always
```

spring.sql.init.mode: always

Spring Boot ejecuta data.sql también con PostgreSQL.

defer-datasource-initialization: true

Espera a que Hibernate haya preparado las tablas antes de ejecutar los INSERT.

Orden de arranque

Hibernate prepara las tablas → Spring Boot ejecuta data.sql
→ arranca la aplicación.

data.sql se ejecuta en cada arranque

Los datos deben tolerar insertarse varias veces sin fallar. En la Actividad 1.5 lo usarás para probar la paginación.

Ideas clave

1

El mapeo se declara con anotaciones sobre la entidad; el XML es el enfoque legado.

2

Estados: transient (sin relación con la BD) → managed (se sincroniza solo) → detached (ya no) / removed (marcado para borrar).

3

«Instalar» Hibernate en Spring Boot es añadir `spring-boot-starter-data-jpa`.

4

IDENTITY delega el id en la base de datos; SEQUENCE reserva ids por adelantado. precision/scale fijan el tipo de la columna.

5

LAZY difiere la carga de una relación hasta que se pide; EAGER la trae siempre.

6

cascade y orphanRemoval propagan borrados y limpian huérfanos automáticamente.

7

ddl-auto es configuración del ORM, no del conector; show-sql y format_sql muestran el SQL real.

8

Para data.sql sobre PostgreSQL:
spring.sql.init.mode: always y defer-datasource-initialization: true.